



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ _____ «СПЕЦИАЛЬНОЕ МАШИНОСТРОЕНИЕ» _____

КАФЕДРА _____ «РАКЕТНЫЕ И ИМПУЛЬСНЫЕ СИСТЕМЫ» (СМ-6) _____

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

ПО ДИСЦИПЛИНЕ:
Компьютерное моделирование

НА ТЕМУ:
Моделирование лесного пожара

Выполнили: студенты группы _____ СМ6-91 _____

_____ Акаёмов А.Е.
_____ Кузнецов А.К.
(подпись, дата) (И.О. Фамилия)

Проверил _____

_____ Федулов В.А.
(подпись, дата) (И.О. Фамилия)

Москва, 2026 г.

Оглавление

Введение	3
Теоретическое описание модели лесного пожара.....	4
Программная реализация игры «Лесной пожар»	7
Эксперименты и результаты.	8
Заключение	15
Список использованных источников	17
Приложение	18

Введение

Клеточный автомат представляет собой математическую модель, в которой пространство разбито на отдельные ячейки, а время течёт дискретными шагами. Каждая ячейка в каждый момент находится в одном из нескольких возможных состояний. На следующем шаге её состояние определяется по единому правилу, учитывающему состояния самой ячейки и её окружения. Такая конструкция позволяет описывать сложные пространственно-распределённые процессы через предельно простые локальные взаимодействия.

Особый интерес вызывают вероятностные клеточные автоматы, в которых правило перехода содержит случайную составляющую. Они позволяют моделировать системы, где наряду с детерминированными процессами присутствуют случайные события: пожары, эпидемии, рост популяций. Модель лесного пожара – один из классических примеров такого подхода. В ней сочетаются детерминированное распространение огня и стохастические процессы восстановления леса и возникновения новых очагов возгорания.

Цель работы – разработать программную реализацию вероятностного клеточного автомата «Лесной пожар» на языке Python, исследовать его поведение при различных параметрах и получить количественные характеристики динамики системы.

Для достижения цели были поставлены следующие задачи:

1. Изучить теоретические основы вероятностных клеточных автоматов.
2. Разработать архитектуру программы, разделив её на логические этапы.
3. Реализовать правило перехода и механизм синхронного обновления поля.
4. Создать визуализацию состояний леса и анимацию процесса.
5. Провести серию экспериментов с различными параметрами модели.
6. Проанализировать полученные результаты и сделать выводы.

Теоретическое описание модели лесного пожара

Классический клеточный автомат задаётся четвёркой:

(A, S, N, F)

где:

- **A** – клеточное пространство, то есть регулярная решётка ячеек;
- **S** – конечное множество состояний, в которых может находиться клетка;
- **N** – окрестность клетки, то есть набор ячеек, влияющих на её состояние;
- **F** – правило перехода, определяющее новое состояние клетки на основе состояний её окрестности.

Время в клеточном автомате дискретно ($t = 0, 1, 2, \dots$), а все клетки обновляются одновременно – синхронно, на основе состояний предыдущего такта. Это ключевое свойство: результаты вычислений не влияют на текущий такт, они применяются только на следующем шаге.

Любой клеточный автомат обладает набором характерных свойств:

1. Локальность – новое состояние клетки зависит только от её ближайшего окружения.
2. Однородность – правило перехода одинаково для всех клеток поля.
3. Синхронность – все клетки обновляются одновременно.
4. Дискретность – пространство, время и множество состояний дискретны.
5. Детерминированность – в базовом случае новое состояние определяется однозначно; в вероятностных моделях допускается случайная составляющая.

Модель лесного пожара – это вероятностный клеточный автомат, описывающий динамику лесного массива. В отличие от детерминированных моделей, здесь переходы между состояниями происходят с заданными вероятностями.

В модели используется три состояния:

$$S = \{0, 1, 2\}$$

где:

- 0 – пустая клетка (выгоревшая территория) – отображается чёрным цветом;
- 1 – зелёное дерево – отображается зелёным цветом;
- 2 – горящее дерево – отображается оранжево-красным цветом.

Три состояния – минимально необходимое множество для описания полного цикла: рост – горение – выгорание – восстановление.

В работе рассматриваются два типа окрестности:

- Окрестность Мура – 8 соседей (четыре ортогональных и четыре диагональных);
- Окрестность фон Неймана – 4 соседа (только ортогональные).

Выбор окрестности существенно влияет на скорость и форму распространения огня: окрестность Мура даёт более быстрое распространение с «круглым» фронтом, окрестность фон Неймана – более медленное с «ромбовидным» фронтом. (рис. 1)

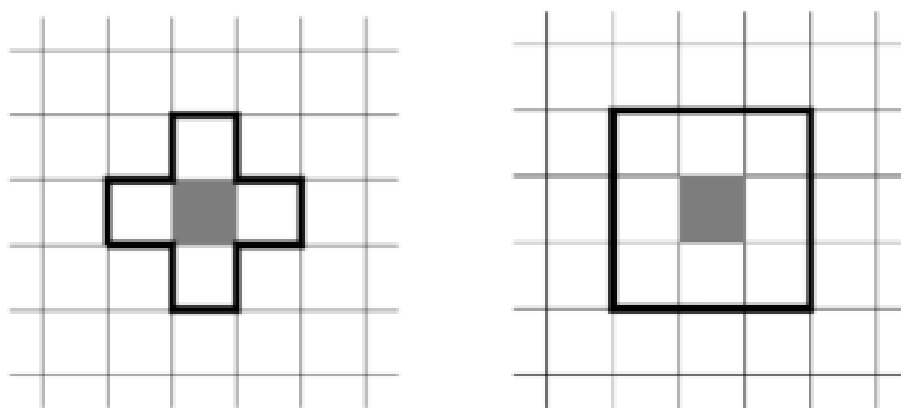


Рис. 1 – Примеры окрестностей:

слева – окрестность Мура; справа – окрестность фон Неймана

Правило перехода применяется ко всем клеткам одновременно и состоит из трёх независимых условий:

Таблица 1. Правила модели лесного пожара

Текущее состояние	Условие	Новое состояние	Смысл
2 (горит)	всегда	0 (пусто)	Дерево сгорает за один такт
1 (дерево)	$k \geq K$ или $\text{random()} < f$	2 (горит)	Возгорание от соседа или молнии
1 (дерево)	иначе	1 (дерево)	Дерево остаётся живым
0 (пусто)	нет горящих соседей и $\text{random()} < p$	1 (дерево)	Вырастает новое дерево
0 (пусто)	иначе	0 (пусто)	Клетка остаётся пустой

Здесь:

- k – число горящих соседей клетки;
- K – порог возгорания (по умолчанию 1);
- f – вероятность удара молнии (самовозгорания);
- p – вероятность роста нового дерева.

Формально правило можно записать так:

$s'(i,j) = 0$, если $s(i,j) = 2$

$s'(i,j) = 2$, если $s(i,j) = 1$ и $(k(i,j) \geq K$ или $\text{random()} < f)$

$s'(i,j) = 1$, если $s(i,j) = 0$ и $k(i,j) = 0$ и $\text{random()} < p$

$s'(i,j) = s(i,j)$, во всех остальных случаях

В модели используются фиксированные граничные условия: клетки за пределами поля считаются пустыми. Это означает, что огонь не может «выйти» за границы поля, а деревья не могут вырасти из «пустоты» за границей. Такой выбор обоснован физической интерпретацией: рассматривается изолированный участок леса.

В зависимости от параметров p , f и K модель демонстрирует разные режимы:

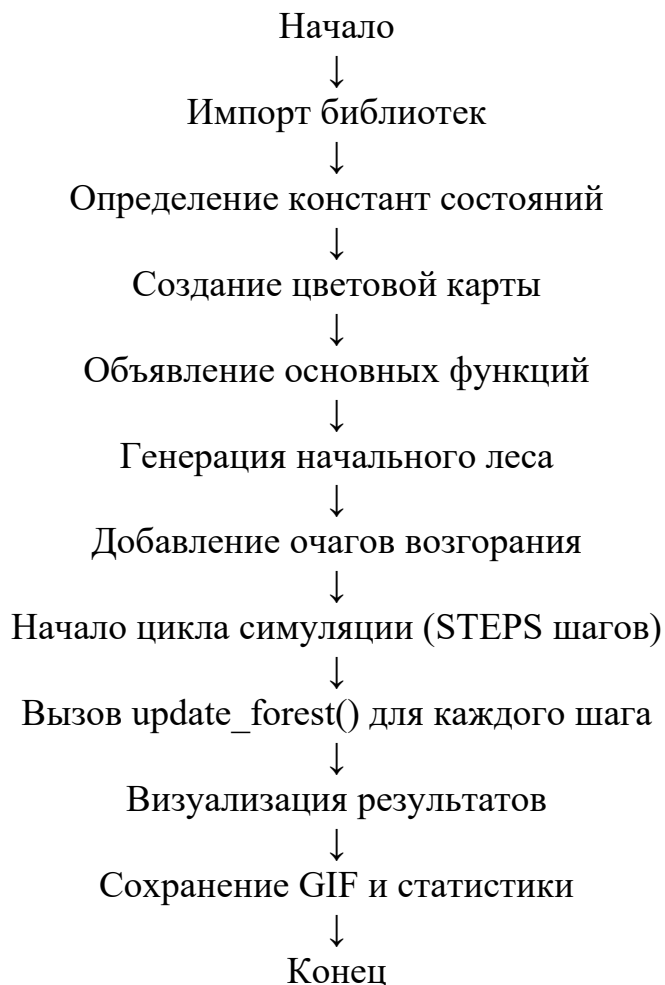
- При малых p и f – лес быстро выгорает и остаётся пустым.
- При больших p – лес быстро восстанавливается, пожары становятся локальными.
- При промежуточных значениях – устанавливается динамическое равновесие, при котором площадь леса колеблется вокруг среднего значения.

Программная реализация игры «Лесной пожар»

Для реализации использован язык Python с подключением следующих библиотек:

- NumPy – для работы с двумерными массивами и векторных вычислений;
- Matplotlib – для визуализации состояний поля и создания анимации;
- Matplotlib.animation – для покадрового обновления изображения;
- Matplotlib.colors – для создания пользовательской цветовой карты.

Далее представлена блок-схема работы программы:



Эксперименты и результаты.

Сценарий 1:

Начальное поле заполняется деревьями с плотностью 0.5 (50% клеток - деревья). Затем вручную добавляются три очага возгорания в разных частях поля.

На рисунке видно равномерное распределение зелёных деревьев по всему полю с тремя очагами возгорания (красные точки).

Через несколько десятков тактов огонь распространяется от очагов возгорания, образуя расширяющиеся «волны» красного цвета. Деревья перед фронтом огня загораются, а позади остаётся чёрная выгоревшая территория.

После того как пожар затухает, начинается фаза восстановления: на пустых клетках с вероятностью $p = 0.01$ вырастают новые деревья. Постепенно лес восстанавливается, но уже в другой конфигурации.

На рисунке 2 представлен сценарий 1.

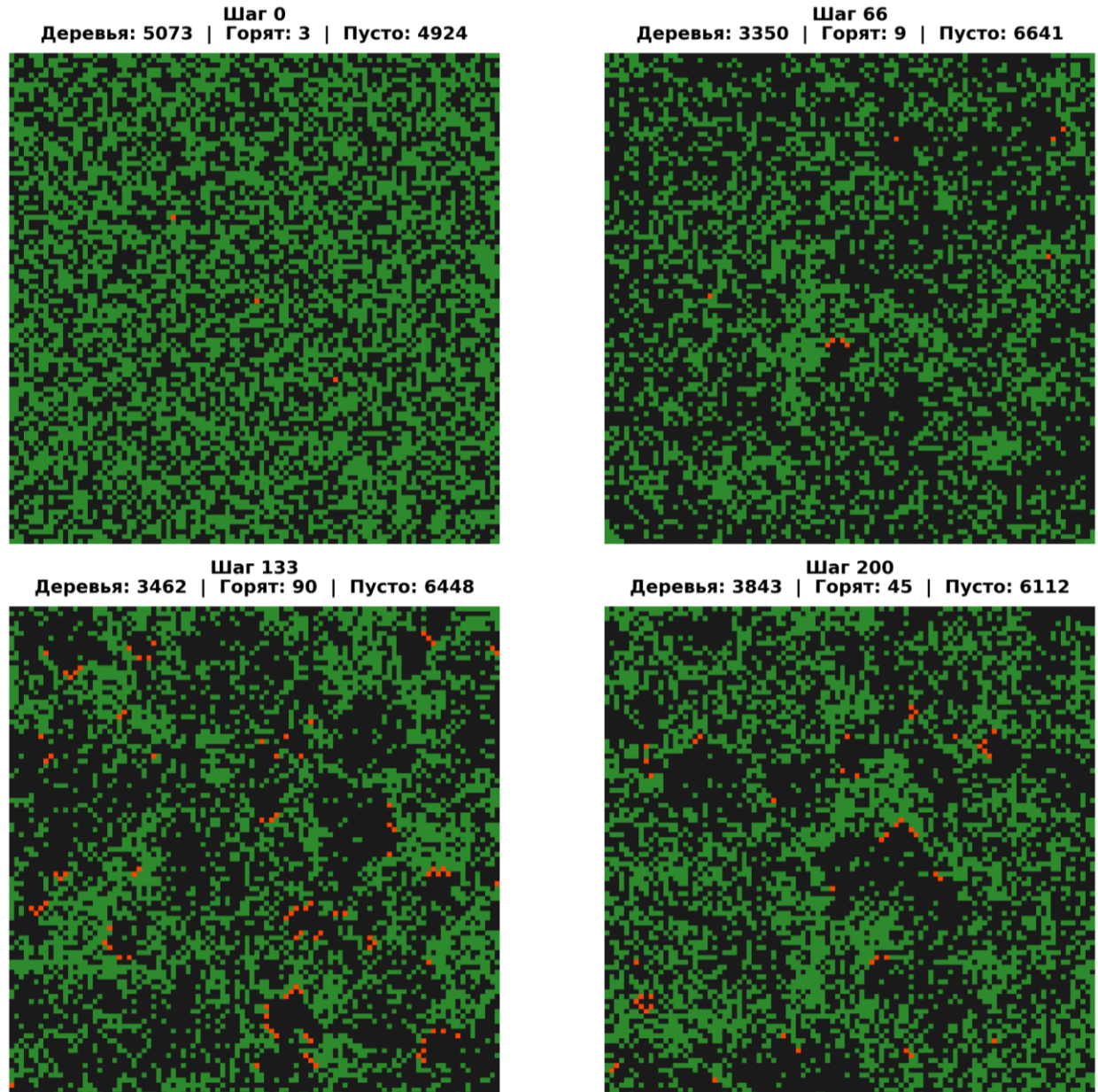


Рис. 2 – Случайный лес с очагами возгорания

Сценарий 2:

Этот сценарий демонстрирует, как начальная плотность деревьев влияет на динамику пожара.

При высокой плотности (80%) огонь распространяется очень быстро, и почти весь лес выгорает за несколько десятков тактов.

При низкой плотности (20%) огонь распространяется медленно, часто затухает сам по себе, не доходя до границ поля, так как горящие деревья не имеют достаточного количества соседей для передачи огня.

На рисунке 3,4 представлен сценарий 2.

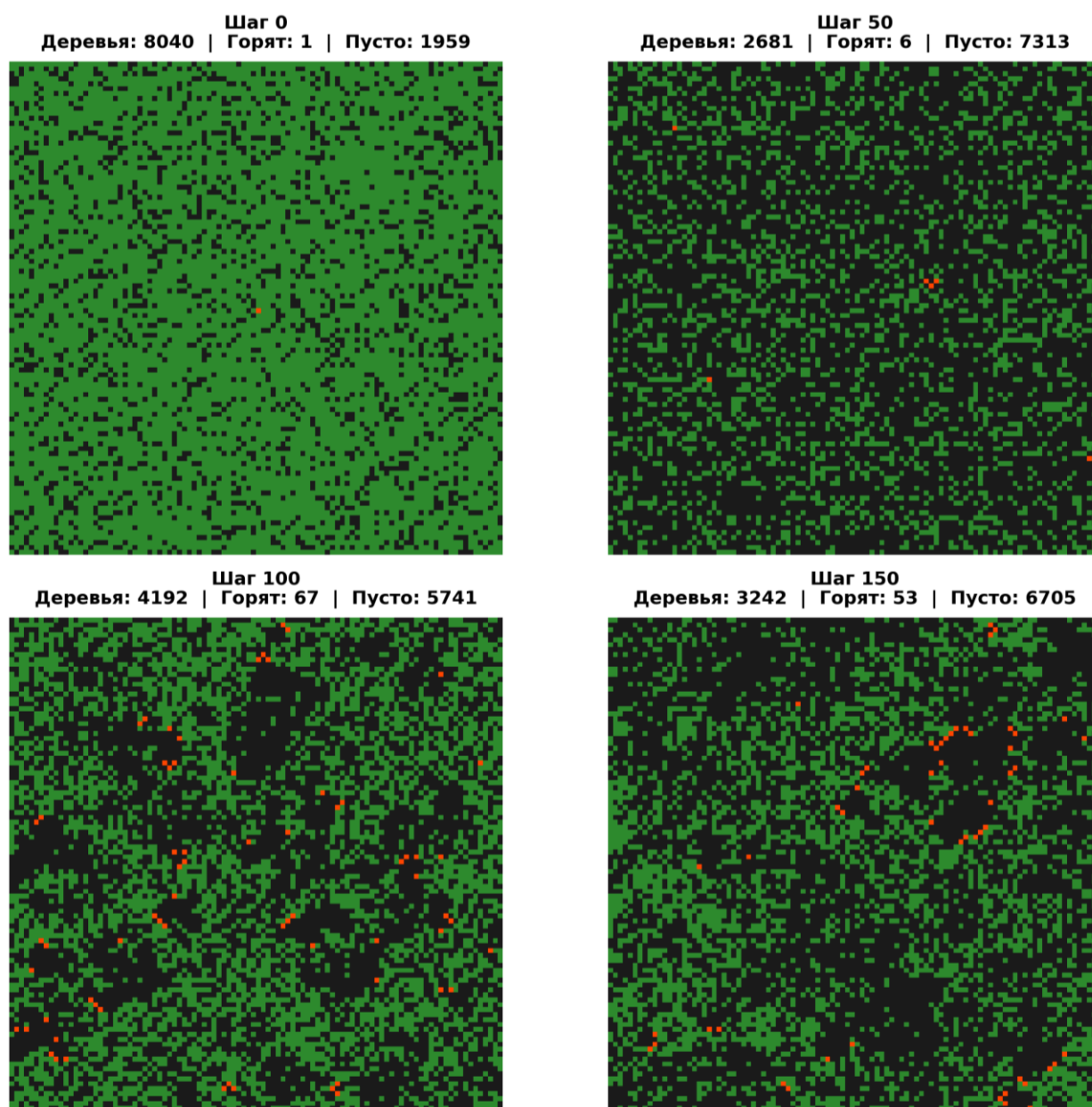


Рис. 3 – Высокая плотность деревьев (80%)

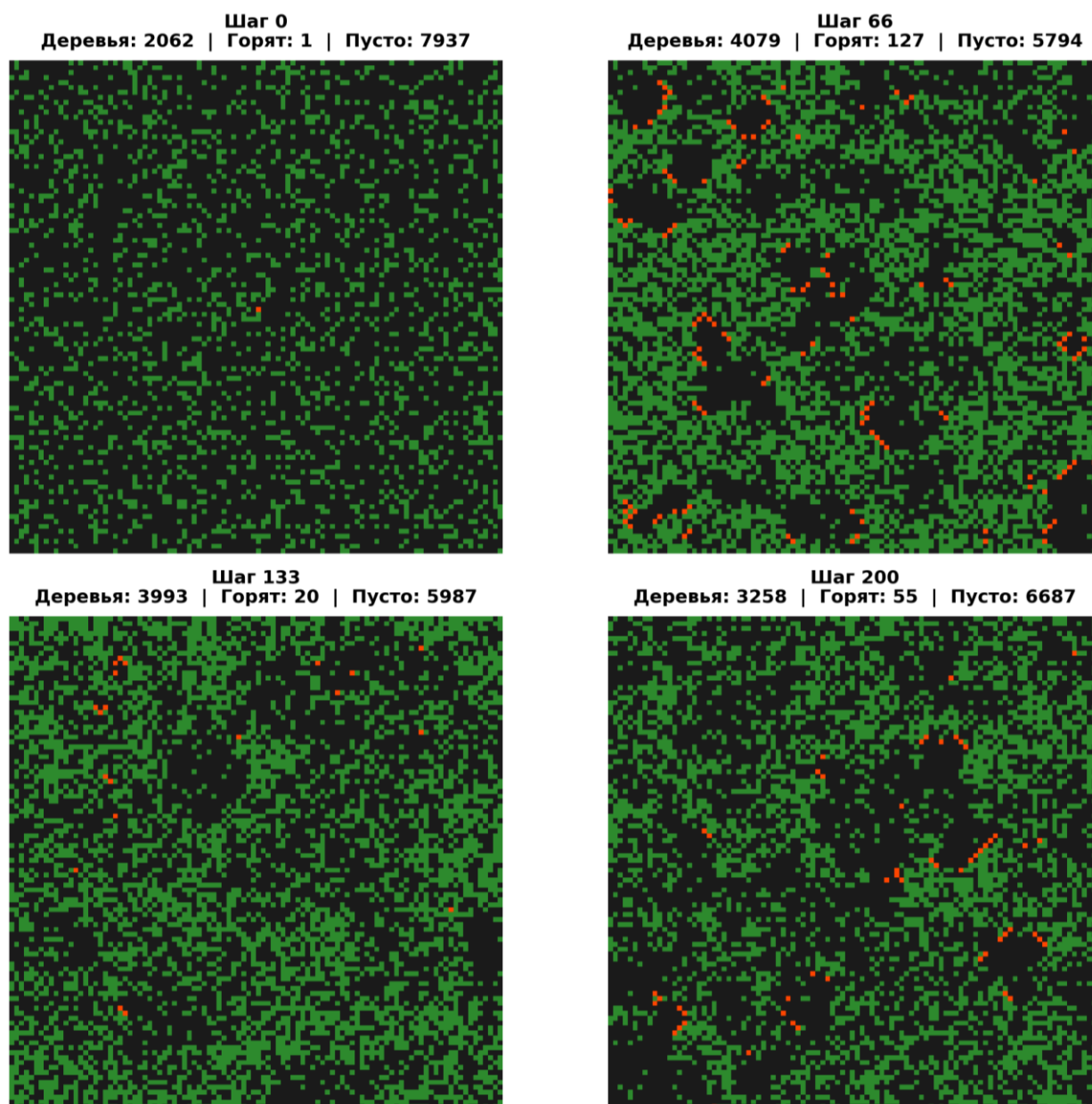


Рис. 4 – Низкая плотность деревьев (20%)

Сценарий 3:

Этот сценарий демонстрирует роль параметра p – вероятности роста нового дерева на пустой клетке.

При $p = 0$ лес не восстанавливается: после пожара поле остаётся пустым навсегда. Это модель «невосстанавливаемого» леса.

При $p = 0.05$ лес восстанавливается быстро, и пожары становятся локальными: огонь не успевает распространиться, так как деревья вокруг ещё не выросли.

При промежуточных значениях ($p \approx 0.01$) устанавливается динамическое равновесие: лес постоянно горит и восстанавливается, площадь деревьев колеблется вокруг среднего значения.

На рисунке 5 представлен сценарий 3.

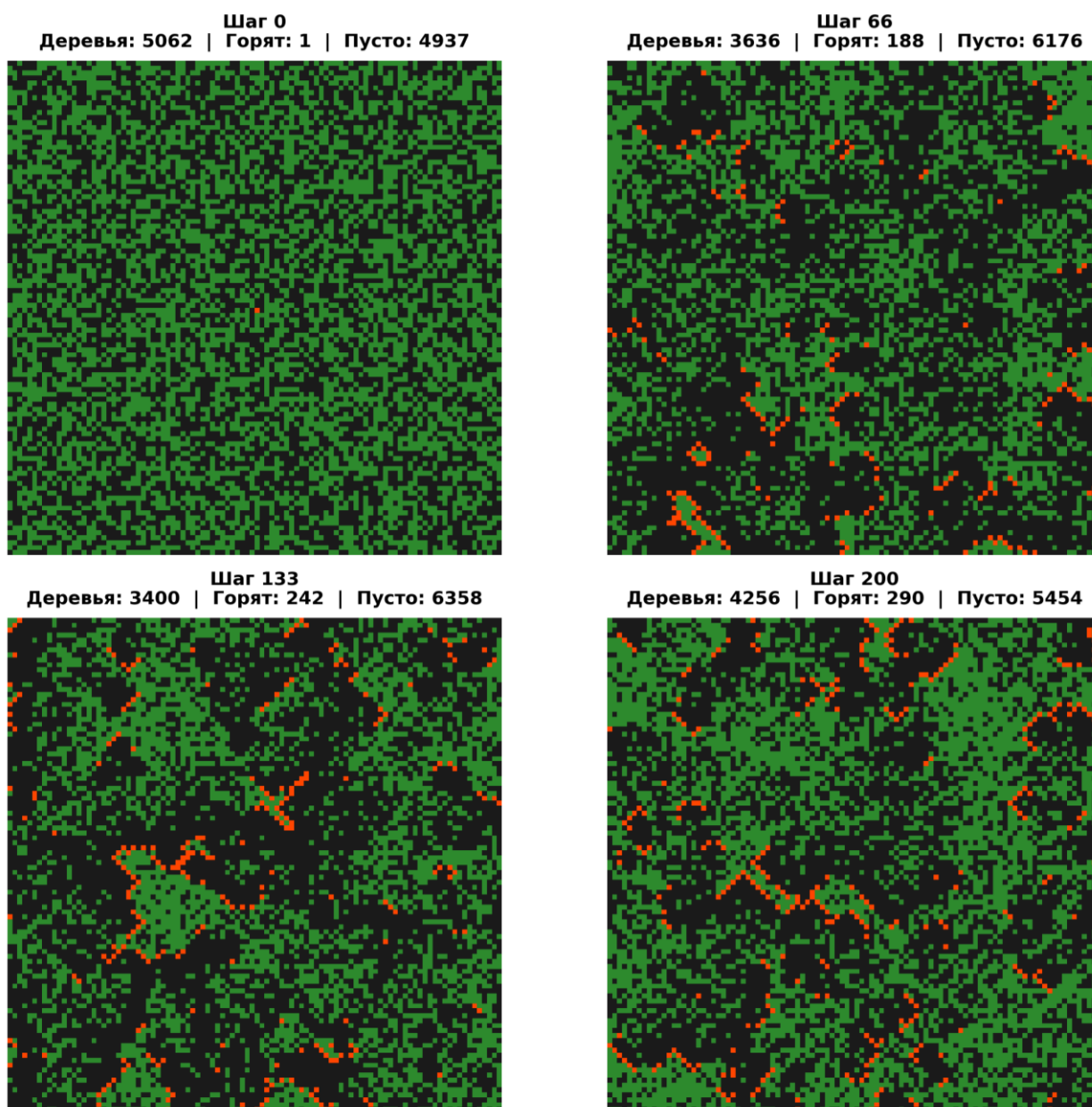


Рис. 5 – Быстрый рост деревьев ($P=0,05$)

Сценарий 4:

Этот сценарий демонстрирует роль параметра f – вероятности самовозгорания дерева от удара молнии.

При $f = 0$ пожары возникают только от вручную добавленных очагов и быстро затухают. Лес восстанавливается и остаётся стабильным.

При $f > 0$ пожары возникают случайно в разных местах поля, что приводит к постоянному «мерцанию» огня и более сложной динамике.

На рисунке 6 представлен сценарий 4.

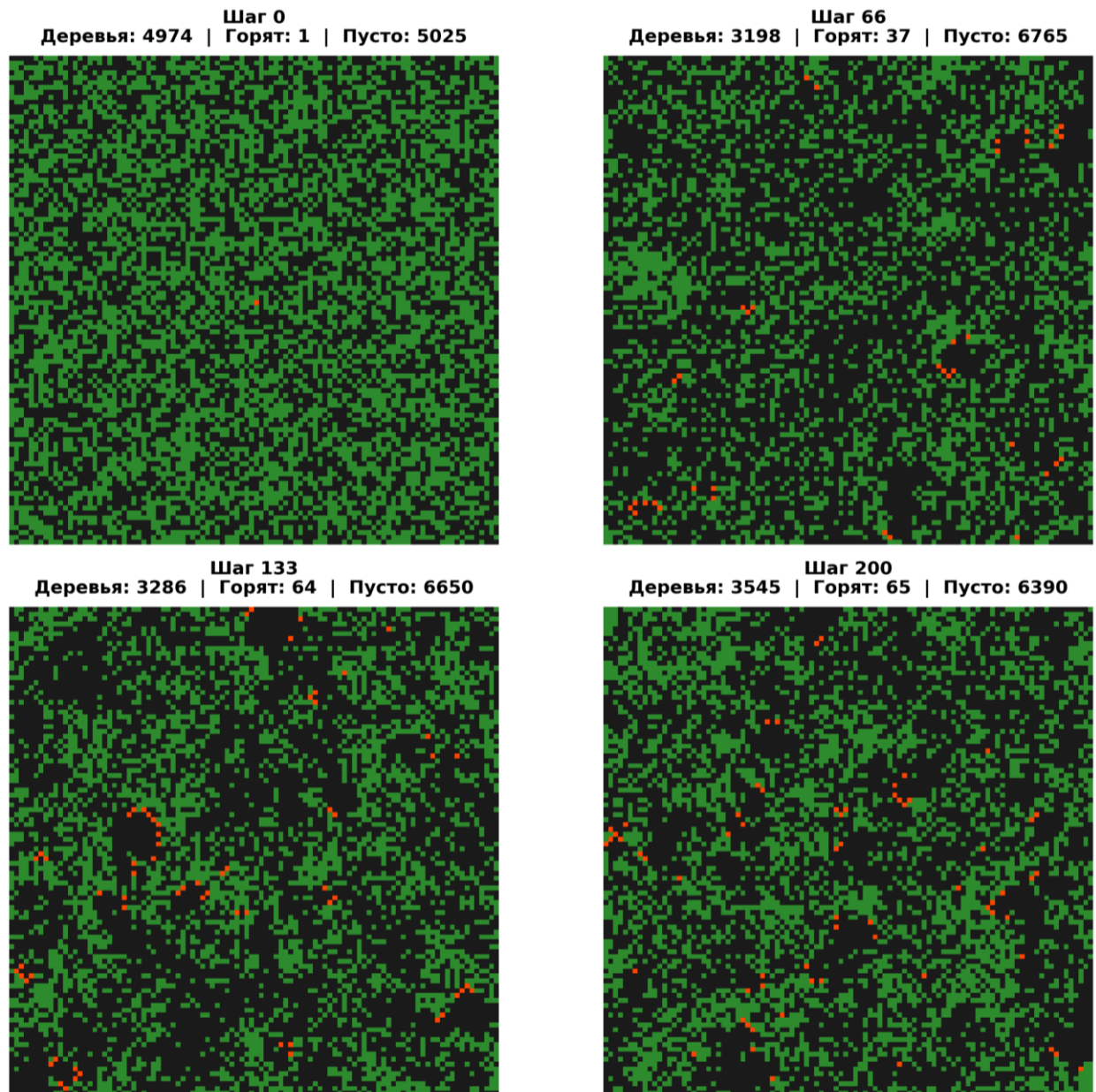


Рис. 6 – Удары молнии ($f=0,001$)

Сценарий 5:

Этот сценарий демонстрирует разницу между окрестностью Мура и фон Неймана.

При окрестности Мура огонь распространяется быстрее и образует «круглые» фронты. За то же время выгорает большая площадь.

При окрестности Неймана огонь распространяется медленнее и образует «ромбовидные» фронты. Пожары чаще затухают сами по себе.

На рисунке 7,8 представлен сценарий 5.

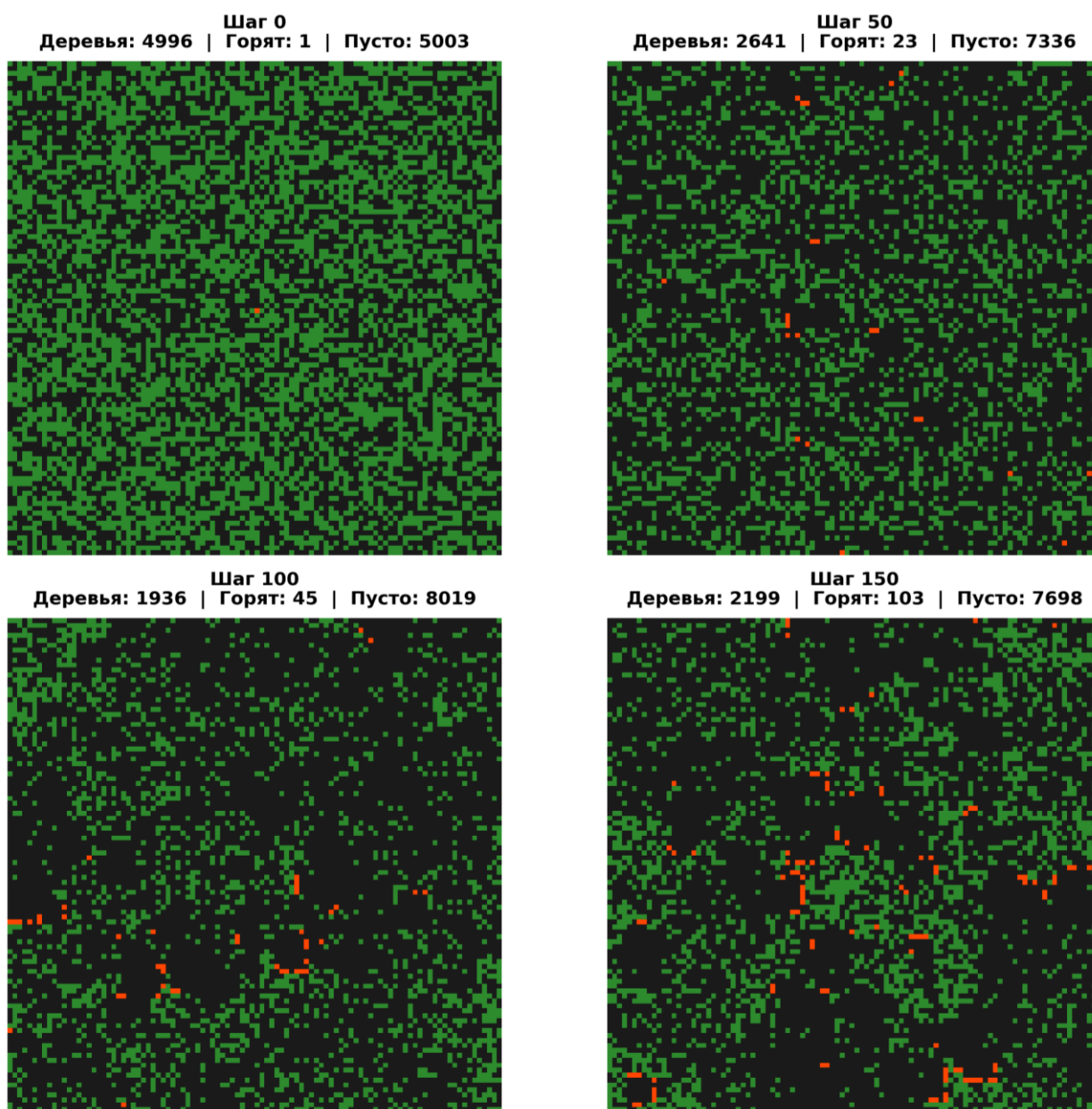


Рис. 7 – Окрестность Мура

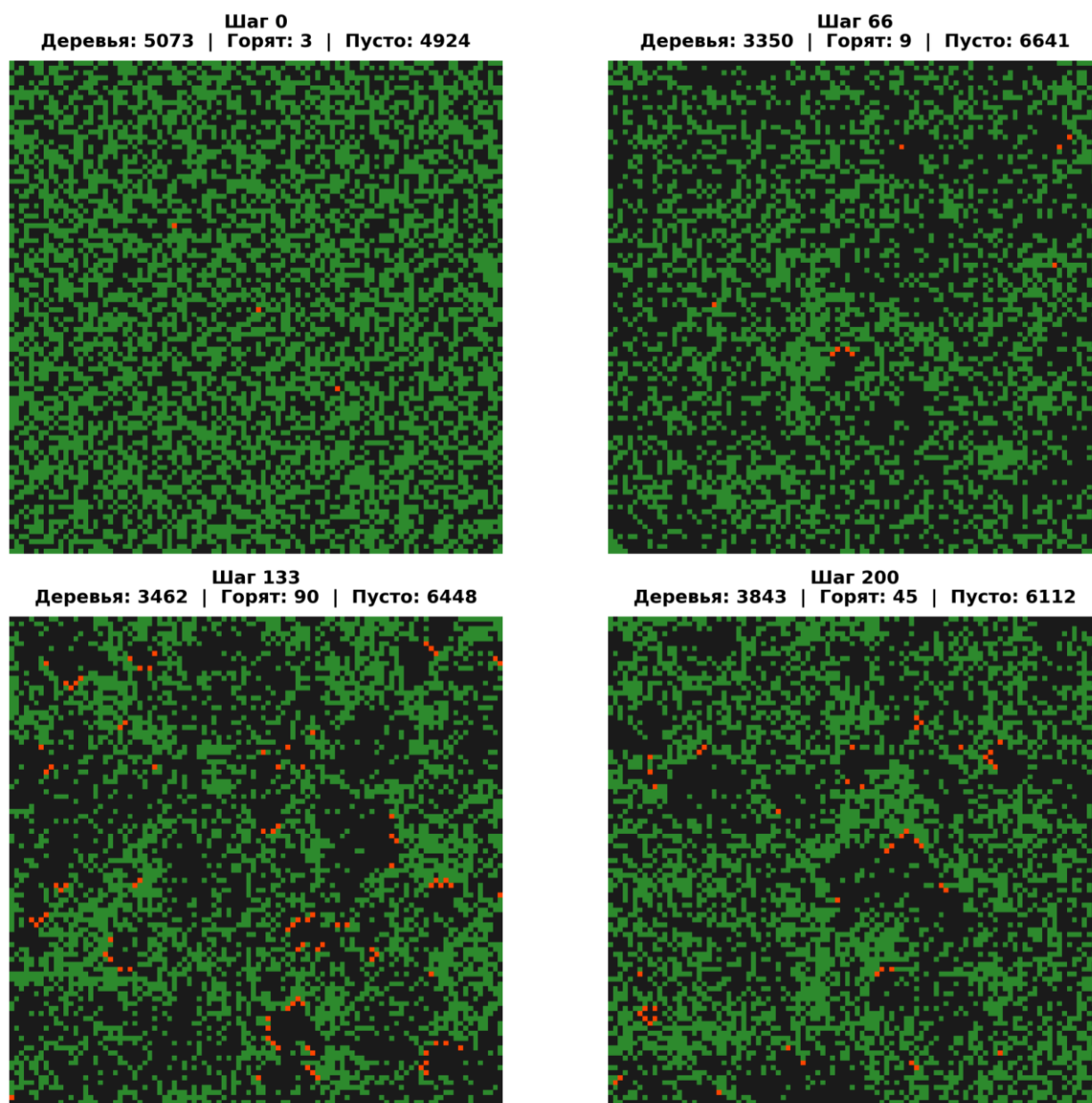


Рис. 8 – Окрестность Неймана

Заключение

В ходе работы был разработан, протестирован и исследован вероятностный клеточный автомат «Лесной пожар» на языке Python.

Были исследованы различные режимы поведения модели в зависимости от параметров p (вероятность роста), f (вероятность удара молнии) и типа окрестности. Показано, что при промежуточных значениях параметров система приходит к динамическому равновесию, характерному для самоорганизованной критичности.

Реализация использует векторные операции NumPy и `np.roll` для подсчёта соседей, что обеспечивает высокую скорость вычислений даже при больших размерах поля. Модель может быть легко расширена: добавление ветра, неоднородности ландшафта, различных типов растительности и других факторов.

Список использованных источников

1. Тоффли Т., Марголюс Н. Машины клеточных автоматов. – М.: Мир, 1991. – 280 с.
2. Вольфрам С. Новая наука. – М.: URSS, 2021. – 728 с.
3. Drossel B., Schwabl F. Self-organized critical forest-fire model // Physical Review Letters. – 1992. – Vol. 69, № 11. – P. 1629–1632.
4. NumPy Documentation. [Электронный ресурс]. –
URL: <https://numpy.org/doc/stable/> (дата обращения: 26.09.2026).
5. Matplotlib Documentation. [Электронный ресурс]. –
URL: <https://matplotlib.org/stable/> (дата обращения: 26.09.2026).
6. SciPy Documentation. [Электронный ресурс]. –
URL: <https://docs.scipy.org/doc/scipy/> (дата обращения: 26.09.2026).

Приложение

```
import numpy as np
import matplotlib
matplotlib.use('Agg') # Чтобы окна не открывались при массовой генерации
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from matplotlib.animation import FuncAnimation
import os

# =====
# Константы состояний
# =====
EMPTY = 0
TREE = 1
BURNING = 2

fire_cmap = ListedColormap(['#1a1a1a', '#2d8a2d', '#ff4500'])

# Папка для выходных файлов
OUTPUT_DIR = "forest_fire_output"
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Качество картинок
DPI = 200

# =====
# Генерация начального леса
# =====
def create_forest(size, tree_density=0.4, seed=None):
    if seed is not None:
        np.random.seed(seed)
    rows, cols = size
    forest = np.zeros((rows, cols), dtype=int)
    random_values = np.random.random((rows, cols))
    forest[random_values < tree_density] = TREE
    return forest

# =====
# Подсчёт горящих соседей (ТОЛЬКО фон Нейман)
# =====
def count_burning_neighbors(forest):
    """
    Подсчёт горящих соседей в окрестности фон Неймана (4 соседа):
    сверху, снизу, слева, справа. Диагонали не учитываются.
    """
    burning_mask = (forest == BURNING).astype(int)
    count = np.zeros_like(forest, dtype=int)

    # Только 4 ортогональных направления
    offsets = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for di, dj in offsets:
        shifted = np.roll(burning_mask, shift=(di, dj), axis=(0, 1))
        # Фиксированные границы: обнуляем завёрнутые края
        if di == -1: shifted[-1, :] = 0
```

```

        elif di == 1: shifted[0, :] = 0
        if dj == -1: shifted[:, -1] = 0
        elif dj == 1: shifted[:, 0] = 0
        count += shifted

    return count

# =====
# Один такт модели
# =====
def update_forest(forest, p, f, k=1):
    """
    Один такт модели лесного пожара.
    Используется окрестность фон Неймана (4 соседа).
    """
    new_forest = forest.copy()
    burning_neighbors = count_burning_neighbors(forest)

    is_empty = (forest == EMPTY)
    is_tree = (forest == TREE)
    is_burning = (forest == BURNING)

    # Правило 1: горящее дерево → пусто
    new_forest[is_burning] = EMPTY

    # Правило 2: возгорание
    ignites_by_neighbor = is_tree & (burning_neighbors >= k)
    lightning = is_tree & (np.random.random(forest.shape) < f)
    new_forest[ignites_by_neighbor | lightning] = BURNING

    # Правило 3: рост нового дерева
    can_grow = is_empty & (burning_neighbors == 0)
    grows = can_grow & (np.random.random(forest.shape) < p)
    new_forest[grows] = TREE

    return new_forest

# =====
# Симуляция
# =====
def simulate_forest_fire(initial_forest, steps, p, f, k=1):
    history = [initial_forest.copy()]
    current = initial_forest
    for _ in range(steps):
        current = update_forest(current, p, f, k)
        history.append(current.copy())
    return history

# =====
# Сохранение 4 ключевых кадров (2x2)
# =====
def save_states_image(history, title, filename, steps_to_show=4):
    n_plots = min(steps_to_show, len(history))
    indices = np.linspace(0, len(history) - 1, n_plots, dtype=int)

```

```

cols = 2
rows = 2
fig, axes = plt.subplots(rows, cols, figsize=(12, 12), dpi=DPI)
axes = np.array(axes).flatten()

for idx, ax in enumerate(axes):
    if idx < n_plots:
        step = indices[idx]
        ax.imshow(history[step], cmap=fire_cmap, interpolation='nearest')

        n_empty = int(np.sum(history[step] == EMPTY))
        n_tree = int(np.sum(history[step] == TREE))
        n_burning = int(np.sum(history[step] == BURNING))

        ax.set_title(f'Шаг {step}\n'
                    f'Деревья: {n_tree} | Горят: {n_burning} | Пусто: {n_empty}',
                    fontsize=13, fontweight='bold', pad=10)
        ax.axis('off')
    else:
        ax.axis('off')

plt.suptitle(title, fontsize=18, fontweight='bold', y=0.98)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig(filename, dpi=DPI, bbox_inches='tight', facecolor='white')
plt.close()
print(f" Сохранено изображение: {filename}")

# =====
# Сохранение графика статистики
# =====
def save_statistics_image(history, title, filename):
    steps = len(history)
    n_trees, n_burning, n_empty = [], [], []
    for state in history:
        n_trees.append(int(np.sum(state == TREE)))
        n_burning.append(int(np.sum(state == BURNING)))
        n_empty.append(int(np.sum(state == EMPTY)))

    fig, ax = plt.subplots(figsize=(12, 6), dpi=DPI)
    time = np.arange(steps)
    ax.plot(time, n_trees, label='Деревья', color='#2d8a2d', linewidth=2.5)
    ax.plot(time, n_burning, label='Горят', color='#ff4500', linewidth=2.5)
    ax.plot(time, n_empty, label='Пусто', color='#1a1a1a', linewidth=2.5)
    ax.set_xlabel('Время (шаг)', fontsize=14)
    ax.set_ylabel('Количество клеток', fontsize=14)
    ax.set_title(title, fontsize=16, fontweight='bold')
    ax.legend(fontsize=13)
    ax.grid(True, alpha=0.3)
    ax.tick_params(labelsize=12)
    plt.tight_layout()
    plt.savefig(filename, dpi=DPI, bbox_inches='tight', facecolor='white')
    plt.close()
    print(f" Сохранён график: {filename}")

# =====

```

```

# Сохранение GIF-анимации
# =====
def save_animation(history, title, filename, interval=100):
    fig, ax = plt.subplots(figsize=(8, 8), dpi=100)

    def update(frame):
        ax.clear()
        ax.imshow(history[frame], cmap=fire_cmap, interpolation='nearest')
        n_tree = int(np.sum(history[frame] == TREE))
        n_burning = int(np.sum(history[frame] == BURNING))
        n_empty = int(np.sum(history[frame] == EMPTY))
        total = history[frame].size
        ax.set_title(f'{title}\nВремя: {frame}\n'
                    f'Деревья: {n_tree} ({100*n_tree/total:.1f}%) | '
                    f'Горят: {n_burning} ({100*n_burning/total:.1f}%) | '
                    f'Пусто: {n_empty} ({100*n_empty/total:.1f}%)',
                    fontsize=11)
        ax.axis('off')
        return ax,

    anim = FuncAnimation(fig, update, frames=len(history),
                        interval=interval, blit=False)
    anim.save(filename, writer='pillow', fps=10)
    plt.close()
    print(f" Сохранена анимация: {filename}")

# =====
# Общая функция для запуска сценария
# =====
def run_scenario(name, description, initial_forest, steps, p, f, k):
    print("\n" + "=" * 70)
    print(f" СЦЕНАРИЙ: {name}")
    print(f" {description}")
    print(f" Параметры: p={p}, f={f}, k={k}, окрестность=фон Неймана, шагов={steps}")
    print("=" * 70)

    history = simulate_forest_fire(initial_forest, steps, p, f, k)

    safe_name = name.lower().replace(" ", "_").replace(".", "")
    base = os.path.join(OUTPUT_DIR, safe_name)

    save_states_image(history, f"{name}: 4 ключевых кадра", f"{base}_states.png")
    save_statistics_image(history, f"{name}: динамика", f"{base}_stats.png")
    save_animation(history, name, f"{base}_animation.gif")

    final = history[-1]
    print(f" Финальное состояние: "
          f"деревья={int(np.sum(final == TREE))}, "
          f"горят={int(np.sum(final == BURNING))}, "
          f"пусто={int(np.sum(final == EMPTY))}")

# =====
# СЦЕНАРИЙ 1. Случайный лес с очагами возгорания
# =====
def scenario_1_random_with_fire():

```

```

size = (100, 100)
forest = create_forest(size, tree_density=0.5, seed=42)
forest[size[0]//2, size[1]//2] = BURNING
forest[size[0]//3, size[1]//3] = BURNING
forest[2*size[0]//3, 2*size[1]//3] = BURNING
run_scenario(
    "1. Случайный лес с очагами возгорания",
    "Начальная плотность 50%, три очага возгорания.",
    forest, steps=200, p=0.01, f=0.0005, k=1
)

# =====
# СЦЕНАРИЙ 2. Высокая плотность деревьев
# =====
def scenario_2_high_density():
    size = (100, 100)
    forest = create_forest(size, tree_density=0.8, seed=1)
    forest[size[0]//2, size[1]//2] = BURNING
    run_scenario(
        "2. Высокая плотность деревьев (80%)",
        "Плотный лес – огонь распространяется очень быстро.",
        forest, steps=150, p=0.01, f=0.0005, k=1
    )

# =====
# СЦЕНАРИЙ 3. Низкая плотность деревьев
# =====
def scenario_3_low_density():
    size = (100, 100)
    forest = create_forest(size, tree_density=0.2, seed=2)
    forest[size[0]//2, size[1]//2] = BURNING
    run_scenario(
        "3. Низкая плотность деревьев (20%)",
        "Разреженный лес – огонь часто затухает сам.",
        forest, steps=200, p=0.01, f=0.0005, k=1
    )

# =====
# СЦЕНАРИЙ 4. Быстрый рост деревьев
# =====
def scenario_4_growth_probability():
    size = (100, 100)
    forest = create_forest(size, tree_density=0.5, seed=3)
    forest[size[0]//2, size[1]//2] = BURNING
    run_scenario(
        "4. Быстрый рост деревьев (p=0.05)",
        "Лес восстанавливается быстро, пожары локальны.",
        forest, steps=200, p=0.05, f=0.0005, k=1
    )

# =====
# СЦЕНАРИЙ 5. Удары молнии
# =====
def scenario_5_lightning():

```

```

size = (100, 100)
forest = create_forest(size, tree_density=0.5, seed=4)
forest[size[0]//2, size[1]//2] = BURNING
run_scenario(
    "5. Удары молнии (f=0.001)",
    "Пожары возникают случайно по всему полю.",
    forest, steps=200, p=0.01, f=0.001, k=1
)

# =====
# СЦЕНАРИЙ 6. Медленный рост деревьев
# =====
def scenario_6_slow_growth():
    size = (100, 100)
    forest = create_forest(size, tree_density=0.5, seed=5)
    forest[size[0]//2, size[1]//2] = BURNING
    run_scenario(
        "6. Медленный рост деревьев (p=0.001)",
        "Лес восстанавливается медленно – долго остаётся пустым.",
        forest, steps=200, p=0.001, f=0.0005, k=1
    )

# =====
# ЗАПУСК ВСЕХ 6 СЦЕНАРИЕВ
# =====
if __name__ == "__main__":
    print("=" * 70)
    print(" ЗАПУСК 6 СЦЕНАРИЕВ МОДЕЛИ ЛЕСНОГО ПОЖАРА")
    print(" Окрестность: фон Неймана (4 соседа)")
    print(f" Все результаты будут сохранены в папку: {OUTPUT_DIR}")
    print(f" DPI картинок: {DPI} (высокое качество)")
    print("=" * 70)

    scenario_1_random_with_fire()
    scenario_2_high_density()
    scenario_3_low_density()
    scenario_4_growth_probability()
    scenario_5_lightning()
    scenario_6_slow_growth()

    print("\n" + "=" * 70)
    print(" ВСЕ 6 СЦЕНАРИЕВ ЗАВЕРШЕНЫ")
    print(f" Файлы сохранены в: {os.path.abspath(OUTPUT_DIR)}")
    print("=" * 70)
    print("\nДля каждого сценария создано три файла:")
    print(" *_states.png – 4 ключевых кадра (сетка 2x2)")
    print(" *_stats.png – график динамики деревьев/огня/пустоты")
    print(" *_animation.gif – полная анимация сценария")

```